TASK-4 DBMS

Order Management

```
create database ordermanagement;
use ordermanagement;


create table Customers (
    Customer_ID INT PRIMARY KEY AUTO_INCREMENT,
    Customer_Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Phone VARCHAR(15),
    City VARCHAR(50)
);
select *from Customers;
CREATE TABLE Products (
    Product_ID INT PRIMARY KEY AUTO_INCREMENT,
    Product_Name VARCHAR(100) NOT NULL UNIQUE,
    Price DECIMAL(10,2) NOT NULL,
    Stock INT NOT NULL DEFAULT 0,
    CHECK (Price > 0),
    CHECK (Stock >= 0)
);
Select * from Products;
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY AUTO_INCREMENT,
    Customer_ID INT NOT NULL,
    Order_Date DATE NOT NULL,
    Total_Amount DECIMAL(10,2) NOT NULL DEFAULT 0,
    Order_Status VARCHAR(20) NOT NULL DEFAULT 'Pending',


    FOREIGN KEY (Customer_ID)
```

```
        REFERENCES Customers(Customer_ID),

    CHECK (Total_Amount >= 0),

    CHECK (Order_Status IN
        ('Pending', 'Shipped', 'Delivered', 'Cancelled'))
);
Select * from Orders;
CREATE TABLE Order_Details (
    Order_Detail_ID INT PRIMARY KEY AUTO_INCREMENT,
    Order_ID INT NOT NULL,
    Product_ID INT NOT NULL,
    Quantity INT NOT NULL,
    Price DECIMAL(10,2) NOT NULL,

    FOREIGN KEY (Order_ID)
        REFERENCES Orders(Order_ID)
        ON DELETE CASCADE,

    FOREIGN KEY (Product_ID)
        REFERENCES Products(Product_ID),

    UNIQUE (Order_ID, Product_ID),

    CHECK (Quantity > 0),
    CHECK (Price > 0)
);
Select * from Order_Details;
INSERT INTO Customers
(Customer_Name, Email, Phone, City)
VALUES
```

('Arun', 'arun@gmail.com', '9876543210', 'Chennai'),

('Priya', 'priya@gmail.com', '9876543211', 'Madurai'),

('Kumar', 'kumar@gmail.com', '9876543212', 'Trichy'),

('Divya', 'divya@gmail.com', '9876543213', 'Coimbatore'),

('Ravi', 'ravi@gmail.com', '9876543214', 'Salem');

INSERT INTO Products

(Product_Name, Price, Stock)

VALUES

('Laptop', 50000.00, 20),

('Mouse', 500.00, 100),

('Keyboard', 1000.00, 50),

('Monitor', 12000.00, 15),

('Headphones', 2000.00, 40);

INSERT INTO Orders

(Customer_ID, Order_Date, Total_Amount, Order_Status)

VALUES

(1, '2026-10-01', 51000.00, 'Pending'),

(2, '2026-10-02', 2500.00, 'Shipped'),

(1, '2026-10-03', 12000.00, 'Delivered'),

(3, '2026-10-04', 1500.00, 'Pending'),

(4, '2026-10-05', 52000.00, 'Delivered');

INSERT INTO Order_Details

(Order_ID, Product_ID, Quantity, Price)

VALUES

(1, 1, 1, 50000.00),

(1, 2, 2, 500.00),

(2, 3, 1, 1000.00),

(2, 5, 1, 2000.00),

```
(3, 4, 1, 12000.00),
(4, 2, 3, 500.00),
(5, 1, 1, 50000.00),
(5, 2, 4, 500.00);

SELECT
    c.Customer_ID,
    c.Customer_Name,
    o.Order_ID,
    o.Order_Date,
    o.Order_Status,
    p.Product_Name,
    od.Quantity,
    od.Price,
    (od.Quantity * od.Price) AS Subtotal,
    o.Total_Amount
FROM Customers c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
JOIN Order_Details od
    ON o.Order_ID = od.Order_ID
JOIN Products p
    ON od.Product_ID = p.Product_ID
ORDER BY o.Order_ID;

SELECT
    c.Customer_ID,
    c.Customer_Name,
    o.Order_ID,
    o.Order_Date,
    o.Total_Amount,
```

```
    o.Order_Status
FROM Customers c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
ORDER BY c.Customer_ID, o.Order_Date;

SELECT
    p.Product_ID,
    p.Product_Name,
    o.Order_ID,
    o.Order_Date,
    od.Quantity,
    od.Price,
    (od.Quantity * od.Price) AS Subtotal
FROM Products p
JOIN Order_Details od
    ON p.Product_ID = od.Product_ID
JOIN Orders o
    ON od.Order_ID = o.Order_ID
ORDER BY p.Product_Name;

SELECT
    p.Product_Name,
    COUNT(DISTINCT od.Order_ID) AS Times_Ordered
FROM Products p
LEFT JOIN Order_Details od
    ON p.Product_ID = od.Product_ID
LEFT JOIN Orders o
    ON od.Order_ID = o.Order_ID
    AND o.Order_Status <> 'Cancelled'
GROUP BY p.Product_ID, p.Product_Name;
```

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    COALESCE(SUM(o.Total_Amount), 0) AS Total_Spending
FROM Customers c
LEFT JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
    AND o.Order_Status <> 'Cancelled'
GROUP BY c.Customer_ID, c.Customer_Name;
```